pfi

Python for Fixed Income

COURSE 7 · NLP, RAG, AND LLMS

Reading EDGAR

WEEK 2 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: reading EDGAR

Three documents, as filed

10-K	fiscal 2025, 4,008,734 bytes
Exhibit 4.1	the Boyd indenture, 2021
8-K	two sections of one filing

Today

1. EDGAR's addresses, as strings
2. The SEC's rules for programs
3. sec_headers and edgar_get
4. HTML to text; inline XBRL
5. A 10-K cut into its items
6. The indenture and an 8-K
7. Excel, Git, the AI check

FETCH_LIVE = False: the notebook sends no request; it reads the course's copies.

The rules for real documents

Today's numbers: sizes, positions, and word counts of documents

Allowed names	Never called
a word count of an item	a measure of risk
what the document states	a view on the company
the filer's own heading	a comparison of issuers

Every excerpt has its source line. Company names appear only there, in the data lines, and in the addresses, as authors.

EDGAR's addresses

906553

CIK: the SEC's number for the filer

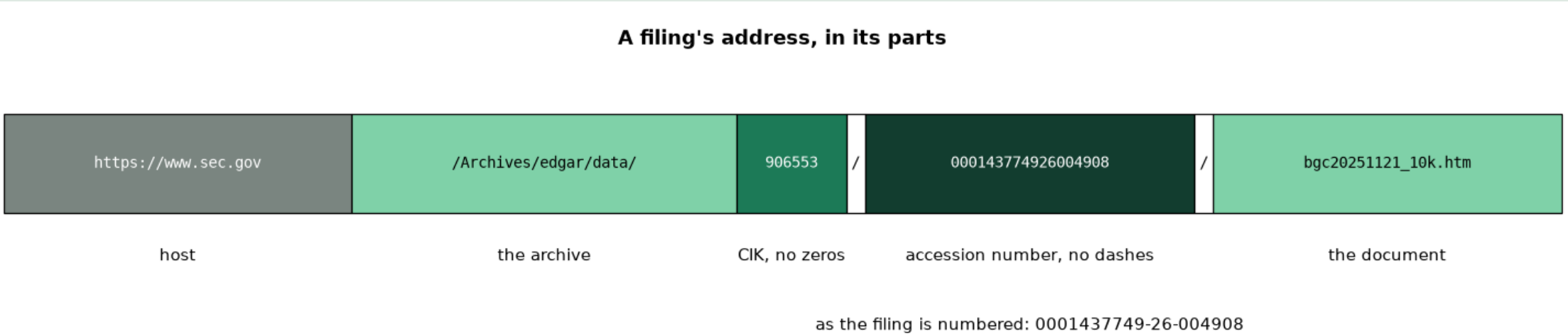
0001437749-26-004908

accession number: one filing

```
https://data.sec.gov/submissions/CIK0000906553.json
https://www.sec.gov/Archives/edgar/data/906553/000143774926004908/
https://www.sec.gov/Archives/edgar/data/906553/000143774926004908/index.json
https://www.sec.gov/Archives/edgar/data/906553/000143774926004908/bgc20251121_10k.htm
```

The CIK padded to ten digits in one, plain in another; dashes dropped in the folder. Build, never retype.

An address, in its parts



7

parts, joined they equal the address

The accession number is written twice: with dashes as the filing is numbered, without them in its folder. Build the address from its parts, never retype it.

"EDGAR started in 1994/1995."

One line, word for word, on the SEC's page "Accessing EDGAR Data", the page that asks programs to declare a user agent. The page says no more about it.

2021	the Boyd indenture, Exhibit 4.1 to an 8-K
2019 to 2025	the 8-K sections in the course's file
2026	the Boyd 10-K for fiscal 2025

Source, read 8 October 2026: sec.gov/search-filings/edgar-search-assistance/accessing-edgar-data

The SEC's rules for programs

A declared user agent

"Please declare your user agent in request headers": a name and an email.

At most 10 requests a second

"... no more than 10 requests per second, regardless of the number of machines"

Copies, cited

Public information, "may be copied or further distributed"; cite the SEC as the source.

Sources, read 8 October 2026: sec.gov/search-filings/edgar-search-assistance/accessing-edgar-data; sec.gov/about/privacy-information

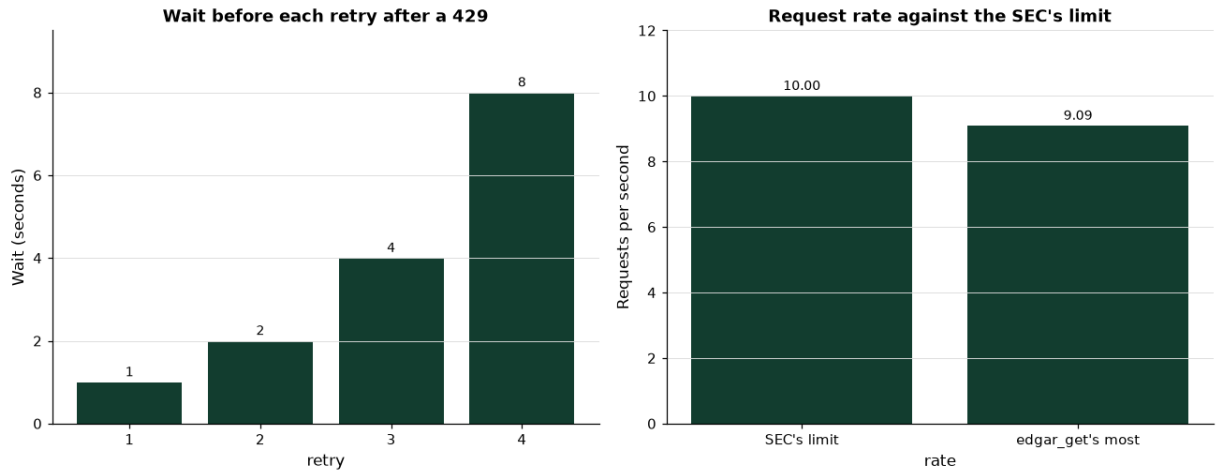
FETCH_LIVE = False in the notebook. Your own name and email go in your own script.

How the fetch functions keep the rules

The problem	Keep the SEC's rules in code, so no request can forget them
sec_headers	Builds the headers; a contact without an email is refused
edgar_get	One GET, sec.gov hosts only, requests spaced under the SEC's limit
Retries	Only where waiting can help: 429 and 503, never a missing page
What comes out	The response text, or an error that says why there is none

sec_headers and edgar_get

```
declared = textkit.sec_headers("Student Name", "student@example.com")
body = textkit.edgar_get(url, declared)   # sec.gov hosts only; 0.11 s apart
```



sec_headers

No default contact. One @ with a . after it, or ValueError.

Retried

429 and 503: wait 1, 2, 4, 8 seconds, then try again.

Never retried

403, 404, and the rest: RuntimeError with the status.

The tests use a fake session and a fake clock: no network, no waiting.

How HTML becomes text

The problem	A filing is HTML, and the course wants its words
Removed	Scripts, styles, and the hidden inline XBRL header
Spacing	White space in the source is one space, as a browser shows it
Lines	Each block tag starts and ends a line; a table cell is one piece of text
What comes out	Plain text, one paragraph per line, to compare in size with the file

Inline XBRL, and what html_to_text keeps

4,008,734
bytes of HTML, as filed

387,293
bytes of the hidden ix:header

2,296
tagged facts in the file

422,118
characters of text

```
soup.find_all(["script", "style", "ix:header"])   # removed, with all they hold
```

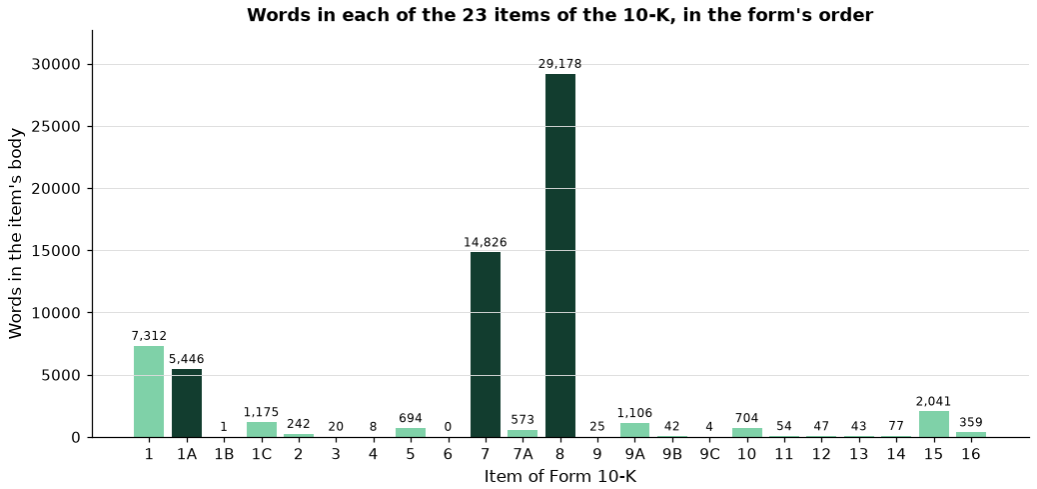
The text is 0.1053 of the file's size, 65,067 words. The header holds passages the visible text holds too: kept, they would count twice. A served copy, with sec.gov's script tag, gives the same text: True.

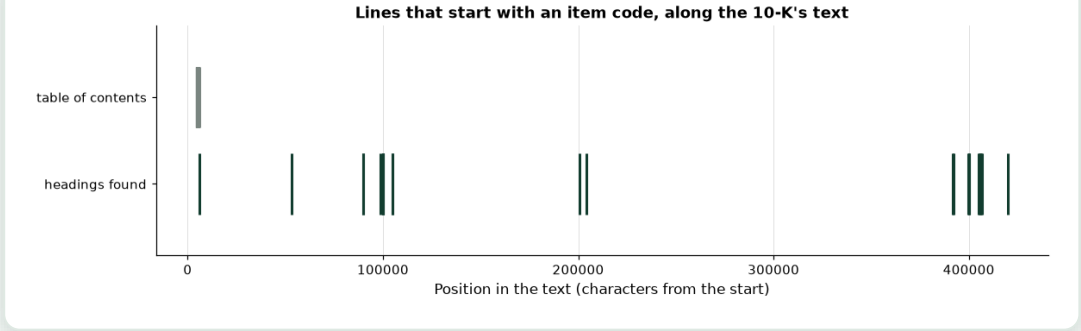
A block tag starts and ends a line; a cell is one piece of text; inline tags add nothing.

How a filing is cut into items

The problem	Find each item of a filing, past the table of contents and cross-references
The search	Each heading, from the last item back to the first: the last one before the next item
Why it works	The contents copies come first; a cross-reference does not start a line
The body	Each item runs to the next heading found
What comes out	One row per item: start, end, words, text, and the number skipped

A 10-K cut into its items





Two traps (23 of 23 items found)

The table of contents: 46 lines start with an item code, 23 of them before Item 1. "ITEM 1A. Risk Factors 11" at character 4,605; the heading at 53,289.

Mentions inside a paragraph: 9 of 55, such as "See Item 8 of Part II". Never cut.

The indenture: the same text, character for character

872,436

bytes: boyd_2021.htm, as filed

392,021

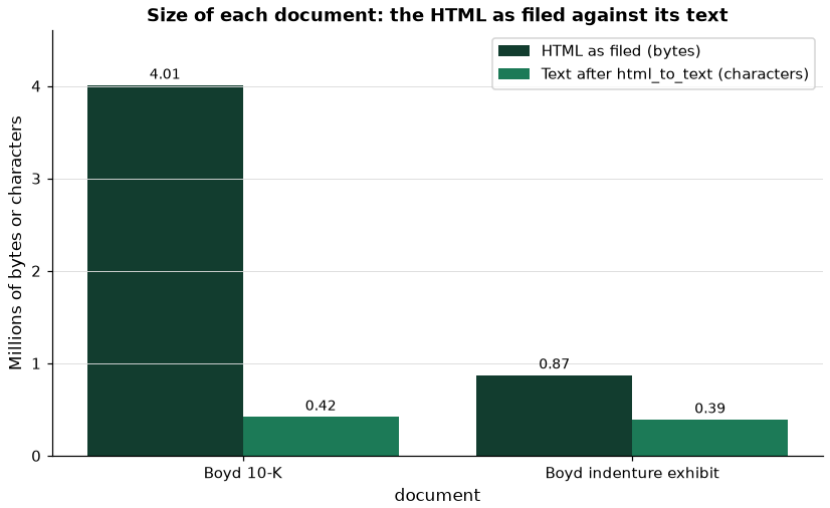
characters of html_to_text

63,595

words

True

equal to boyd_2021.txt




```
assert textkit.html_to_text(exhibit_bytes) ==
  Path("course7_indentures/boyd_2021.txt").read_text(...)
```

Every number about the indenture since day 1 came from this function on this exhibit.

Source: Boyd Gaming Corporation, indenture for the 4.750% Senior Notes due 2031, Exhibit 4.1 to the 8-K filed 2021-06-08, accession 0001193125-21-185495.

8-K items: a cross-reference must not cut

```
Item 1.01 Entry into a Material Definitive Agreement
  ...the stored body...
Item 2.03 Creation of a Direct Financial Obligation ...
  The information contained in Item 1.01 ... in this Item 2.03.
```

157 and 21

words: each body equal to the stored one

5

pieces from a split at every mention

Only a heading that starts a line cuts. The filer's cross-reference stays inside its sentence.

Excel side by side

No HTML parser, no inline XBRL, no request to sec.gov in Excel. Cutting pasted text, yes.

Excel side by side (tested in Excel)

`=TEXTBEFORE(TEXTAFTER(A1,"ITEM 2. Properties"&CHAR(10)),CHAR(10)&"ITEM 3.")`	Item 2's body, equal
`=ROWS(TEXTSPLIT(TRIM(SUBSTITUTE(B2,CHAR(10)," ")),," "))`	242
`=TEXTAFTER(A1,CHAR(10)&"ITEM ",-1)`	after the last heading
`=TEXTBEFORE(TEXTAFTER(A1,"ITEM 3. Legal Proceedings"&CHAR(10)),"Item ")`	"See"
`=TEXTAFTER(A1,"ITEM 7.")`	#N/A

One cell, 1,704 characters: the whole text, 422,118, is over a cell's 32,767.

Save the day: a change, word by word

```
test_textkit.py |  97 ++++++++++++++++++++++++++++++++++++
textkit.py      | 147 ++++++++++++++++++++++++++++++++++++
```

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Units, conventions, and what the function must never do: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Assert what the docstring promises. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the helper sends User-Agent "python-requests" and never uses the caller's name and email.

Code that runs is not the same as code that is right.

What you can do now

- Build EDGAR's addresses from a CIK and an accession
- Keep the SEC's rules in code: a contact, 10 a second
- Turn HTML into text, without the XBRL header
- Cut a 10-K at its items, past the contents
- Cut one item from pasted text in Excel

Week 2, Day 5 Case study: where the models disagree